Semantic Matching in Medieval Latin: A Benchmark, Layerwise Analysis, and Geometric Repair

Anonymous ACL submission

Abstract

Medieval canon law texts preserve some of the earliest sustained records of judicial reasoning in Western Europe, but they survive in manuscript families shaped by spelling variation, fragmentation, and editorial change. As a result, strict semantic matching is a prerequisite for scholarly workflows such as consolidating duplicate passages and linking fragments across witnesses. We introduce a new Medieval Latin dataset of 1,705 canon law fragments grouped into 840 families of meaning-equivalent variants, and evaluate two tasks: duplicate detection and open-set semantic deduplication. We conduct a layerwise analysis of internal representations across a diverse set of Latin-adapted and multilingual models and uncover a striking failure mode. In Latin-adapted T5 encoders, standard mean-pooled hidden states undergo a pronounced mid-layer collapse, causing cosine similarity to lose discriminative power. In contrast, multilingual encoders such as LaBSE and decoder-based models remain substantially more stable. Motivated by this diagnosis, we apply a simple train-free geometric repair based on All-but-the-Top post-processing, which largely removes the mid-layer failure mode and restores semantic separation. Finally, we show that the resulting representations support more reliable instance-level attribution using integrated gradients, enabling an inspectable, scholar-facing semantic matching pipeline.

1 Introduction

Medieval canon law offers a rich record of early judicial reasoning, reflecting evolving ideas about authority, justice, and social order. Yet, these texts do not exist as stable, single versions. Instead, they are preserved through manuscript traditions that introduce variation in wording, fragmentary transmission, and editorial intervention. As a result, passages that express the same legal principle may differ substantially on the surface, while others diverge in subtle but meaningful ways. For scholars, identifying strict semantic equivalence across such variants is a fundamental step in consolidating textual evidence, linking fragments, and tracing the transmission of legal thought.

Figure 1: Cosine similarity distributions for PhilTa at the last layer and at the mid layer, before and after ABTT. At the dip layer. ABTT-based post-processing produces much clearer cosine score separation between semantically similar and different passages.

We study this problem by introducing new data of Medieval Latin canon law passages organized into 840 families of meaning-equivalent variants. We consider two complementary tasks. *(i)* Duplicate detection evaluates whether a pair of passages conveys the same meaning under a strict notion of equivalence. *(ii)* Open-set semantic deduplication reflects corpus-level workflows, requiring models to determine whether a newly observed passage matches any existing passage or represents a previously unseen instance.

A standard approach is to encode passages using pretrained models and compare them with cosine

similarity. While this paradigm is effective for high-resource languages, its behavior in historical languages such as Medieval Latin remains poorly understood. In particular, it remains unclear how semantic information is distributed across model depth and whether commonly used representation extraction methods preserve or distort that information. This motivates a closer examination of model internals rather than treating sentence embeddings as fixed outputs.

To this end, we analyze representations across layers for a range of Latin-adapted and multilingual models. This analysis reveals a consistent failure mode in Latin-adapted T5 encoders: representations obtained via mean pooling lose discriminative structure in intermediate layers, leading to a collapse in semantic separability. Notably, this behavior is not observed to the same extent in multilingual encoders or decoder-based models, which exhibit more stable trends across depth.

This observation points to a geometric explanation. When representations are dominated by a small number of global directions, cosine similarity becomes less sensitive to semantic distinctions, particularly in intermediate layers. We address this issue using a simple, train-free correction based on All-but-the-Top (ABTT) post-processing, which removes dominant components from the embedding space. This correction restores semantic separation and stabilizes performance across layers without modifying model parameters.

A natural approach to both tasks is to embed passages using pretrained models and compare them via cosine similarity. While this paradigm is effective for high-resource languages, its behavior in historical languages such as Medieval Latin remains poorly understood. In particular, standard practice treats sentence representations as fixed outputs, without examining how semantic information is distributed across model depth or how representation extraction affects downstream performance. This raises a key question: to what extent do failures in semantic matching arise from limitations of the underlying model, as opposed to how its internal representations are used?

To answer this, we conduct a layerwise analysis of internal representations across a diverse set of models, including Latin-adapted encoders and multilingual models. We uncover a striking and consistent failure mode. In Latin-adapted T5 encoders, standard mean-pooled hidden states undergo a pronounced mid-layer collapse, in which cosine similarity loses discriminative power and semantic separation degrades sharply, in some cases approaching chance. In contrast, multilingual encoders such as LaBSE and decoder-based models exhibit substantially more stable behavior across depth. This finding shows that model performance on Medieval Latin semantic matching depends critically on how representations are extracted, and that naïve pooling can obscure usable semantic information.

This observation leads to a key insight: the primary bottleneck is not missing semantic content, but the geometry of the representation space induced by common extraction methods. When representations become anisotropic or dominated by a few global directions, cosine similarity fails to distinguish semantically equivalent from non-equivalent passages, particularly in intermediate layers.

Motivated by this diagnosis, we apply a simple, train-free geometric correction based on All-but-the-Top (ABTT) post-processing (Mu and Viswanath, 2018). By removing dominant directions from the embedding space, this approach restores semantic separation and largely eliminates the mid-layer collapse. The resulting representations are more stable across layers and yield consistently stronger performance on both duplicate detection and open-set semantic deduplication, without requiring any task-specific training.

Beyond improving matching performance, geometric correction also enhances interpretability. We find that applying integrated gradients on corrected representations produces clearer and more localized token-level attributions for semantic matches, whereas raw representations yield diffuse and less informative explanations. This enables an inspectable, scholar-facing workflow in which retrieved matches can be both ranked and explained, supporting more reliable use of NLP tools in humanities research.

In summary, this paper makes the following contributions: (1) we introduce a new benchmark for strict semantic matching in Medieval Latin canon law, including both pairwise and open-set formulations; (2) we provide a layerwise analysis revealing a previously unreported mid-layer collapse in Latin-adapted T5 representations under standard pooling; (3) we show that a simple train-free geometric correction restores semantic separability and improves robustness across models and layers, and verify that the dip-and-repair pattern trans-

fers qualitatively to a second, sparser Medieval Latin manuscript collection (decritals); and (4) we demonstrate that improved representation geometry leads to more reliable instance-level attribution, enabling an inspectable semantic matching pipeline for humanities scholars.

2 Task and Dataset

2.1 Task Definition

We study two related tasks.

Task A. Let p_1 and p_2 be two passages and let $\equiv$ denote semantic equivalence, so that $p_1 \equiv p_2$ holds exactly when the two passages express the same meaning. Given an input pair (p_1, p_2), the task is to predict

$$y = \begin{cases} 1 & \text{if } p_1 \equiv p_2, \\ 0 & \text{otherwise.} \end{cases}$$

We score pairs by cosine similarity and classify with a learned threshold τ.

Task B. Given a query passage q, the system scores each directory by the maximum cosine similarity between q and any document already assigned to that directory. We also add a pseudo-option `__NEW__` with score τ for cases where the query should open a new directory. The model then presents the top-k options to a human expert, along with token-level explanations for individual candidate pairs. During evaluation, we simulate that expert by assuming the labeled directory would be selected whenever it appears in the shortlist. We report cumulative top-K accuracy: for each K from 1 to 5, the fraction of queries whose labeled directory appears among the top K options.

2.2 Dataset Construction

This is the part we agreed you'd fill in (professor)

2.3 Train/Test Split

The corpus contains 1,705 medieval Latin manuscript fragments drawn from canon law and organized into 840 directories. Each directory corresponds to one original legal text, and the files inside it are manuscript copies that express the same content. Directory sizes range from 1 to 10 files.

We split the corpus into an 847/858 train/test set with a leak-free protocol (Table 1).[1] The 545 singleton directories are distributed roughly evenly between the two splits at the file level. Doubleton directories (108 in total) are assigned as whole folders, 54 to train and 54 to test, so that both files of each doubleton remain together; this contributes 54 within-directory positive training pairs that a file-level doubleton split would discard. For multi-file directories ($\geq$3 files, 187 directories), we vary the train allocation across folders within each size class: for a class of K folders with directory size n, we distribute train_n uniformly over $\{1, \ldots, n-1\}$ via stratified allocation, with the remainder weighted toward values nearest $n/2$. The class average remains balanced near 50/50, while every size class contributes same-folder positive pairs to training. This avoids the pathological case in which proportional splitting forces every size-3 directory to $(1 \text{ train}, 2 \text{ test})$ and contributes zero same-folder positives from that class. The redesign yields 565 positive training pairs, a roughly 60% increase over the previous proportional split, and a more defensible setup in which no directory size class is structurally excluded from contributing same-folder training positives.

All fitting, including ABTT principal components and threshold tuning, uses training data only. The training set contains 565 positive pairs and the test set contains 595 positive pairs.

Category	Dirs	Train	Test
Singletons (1 file)	545	272	273
Doubletons (2 files)	108	108	108
Multi-file ($\geq$3)	187	467	477
Total	840	**847**	**858**

Table 1: Train and test split statistics under the v2 varied-allocation protocol. Doubleton directories are assigned as whole folders (54 to train, 54 to test); all 108 train doubleton files therefore contribute within-directory positive pairs. The test set contains 535 "existing" files, which have a same-directory partner in test, and 323 "new" files, which do not.

3 Methodology

3.1 Models

We evaluate six pre-trained models from three architecture families:

- **LaTa**: a T5-based seq2seq model pre-trained on Latin (Riemenschneider and Frank, 2023). We extract encoder hidden states (12 transformer blocks).

[1] Implemented in `src/canon_split_v2.py`; deterministic with seed 42.

- **PhilTa**: a philologically-oriented variant of LaTa, also T5-based (Riemenschneider and Frank, 2023).
- **LaBSE**: a language-agnostic BERT sentence encoder covering 109 languages (Feng et al., 2022) (12 encoder layers).
- **mt5-base**: a multilingual T5 encoder-decoder (Xue et al., 2021) (12 encoder layers).
- **Qwen3-0.6B**: a multilingual decoder-only embedding model (28 layers) (Qwen Team, 2025).
- **KaLM-mini**: a multilingual decoder-only embedding model (24 layers) (Lu et al., 2025).

3.2 Embedding Extraction

We extract hidden-state representations from every transformer block, starting at layer 1 and ending at the final block, while skipping layer 0, the static embedding table. LaTa, PhilTa, LaBSE, and mt5-base each contribute 12 layers. Qwen3-0.6B contributes 28, and KaLM-mini contributes 24. Token representations at each layer are then pooled into a single sentence vector, either by plain mean pooling or by SIF weighting.

For the paper-facing pooling baseline, we remove only tokenizer artifacts with no lexical content, such as whitespace, padding, or tokenizer-prefix-only tokens. We keep punctuation and numerals in the pooled representations. In the explanation figures, though, we always show the original decoded tokens rather than filtered display tokens.

3.3 Post-Processing: ABTT

Our main post-processing method is All-but-the-Top (ABTT) (Mu and Viswanath, 2018). We center the pooled sentence embeddings, compute the top-D principal components by SVD on the training set, and project those components out of both train and test embeddings. At each layer we search $D \in \{1, 2, 3, 5, 7, 10\}$ and keep the value that maximizes training-set directory accuracy at rank 1 in Task B. We refer to that as the **optimal** setting.

We also compare against two simpler alternatives. One is SIF-weighted pooling (Arora et al., 2017) without ABTT. The other is PCA whitening (Su et al., 2021). SIF assigns each token w_i the weight $\alpha_i = a/(a + p(w_i))$, where $p(w_i)$ is the unigram probability estimated on the training set and $a = 10^{-3}$.

3.4 Example-Level Attribution

For the explanation figures, we use integrated gradients (Sundararajan et al., 2017) as a local attribution method. Given a query passage and one candidate partner passage, we compute per-sequence token attributions toward pair cosine similarity. The displayed axes always use the original decoded tokens rather than filtered display tokens, even when a token's attribution is close to zero. The heatmap in Figure 4 is therefore a derived pairwise view: token-token cosine similarity is combined with the marginal integrated-gradient scores of the two sequences. We show that view before and after ABTT on the same example. We also render companion self-attention maps for the same pair as supporting context only, since ABTT is applied after encoding and does not change the model's internal attention weights.

3.5 Evaluation Metrics

For Task A, the main measure is **AUCROC**, computed over pairwise cosine similarities, with same-directory pairs treated as positive. We also report **cosine gap**, the difference between the mean same-directory and mean different-directory cosine similarity, as a direct measure of class separation.

For Task B, we report **cumulative top-K accuracy**. For each K from 1 to 5, the metric gives the fraction of test queries whose labeled directory appears among the model's top K options. This directly answers the deployment question: if the expert inspects the top K candidates, how likely are they to find the correct directory? We still use training-set directory accuracy at rank 1 as the model-selection signal for ABTT's D.

3.6 Directory Accuracy vs. Assignment Accuracy

Two closely related Task A numbers appear throughout our results and answer genuinely different questions. We state the distinction explicitly because the two are easy to conflate.

Directory accuracy at rank 1 (Acc@1). For each test file that has at least one same-directory partner elsewhere in the test set, we rank every other test file by cosine similarity and score the query correct if the top-ranked neighbour comes from the same directory. Every such query is forced to produce an answer; there is no "I don't know" option and the threshold τ plays no role. Acc@1

answers a single question: *did we rank the correct item at the top?*

Assignment accuracy. Assignment accuracy adds the complementary question: *did we make the right decision, including knowing when to say "this is new"?* For every test file i we compute $s_i = \max_{j \neq i} \cos(\mathbf{e}_i, \mathbf{e}_j)$ against the rest of the test set and compare it to the threshold τ, which is learned on the training split by sweeping the F_1-optimal cut on same-directory versus different-directory pairs (no test data is used to fit τ). The decision is binary first,

$$\hat{y}_i = \begin{cases} \text{“existing”} & \text{if } s_i \geq \tau, \\ \text{“new”} & \text{if } s_i < \tau, \end{cases}$$

and a file is scored correct iff this binary prediction matches ground truth (i.e. whether the file actually has a same-directory partner in the test set). Only on files that we predict as “existing” do we additionally look at the top-1 neighbour to recover the predicted directory. We report the overall rate as **overall assignment accuracy** and decompose it into **existing accuracy**, restricted to files whose true partner is in the test set, and **new accuracy**, restricted to files that should have been flagged as novel.

In short, Acc@1 asks “did we rank it correctly”, while assignment accuracy asks “did we make the right decision, including abstaining when appropriate”: the two numbers can diverge sharply whenever τ is poorly calibrated or the novel-file subset is large.

The query-vs-directory variant (`qvd_*` columns in `phase_resubmit_results.csv`) applies exactly the same threshold rule after max-pooling similarities at the directory level instead of the file level, matching the deployment setting of Task B.

4 Results

4.1 Task A: Directory Assignment

Table 2: Per-layer Task A assignment metrics for each model, comparing `baseline` (mean pooling, no correction) against `sif_abtt_optimal` (SIF weighting plus ABTT with D tuned per layer on the train split). Rows in bold mark the best layer per model, selected by ABTT overall assignment accuracy.

		Task A: Directory Assignment					
Model	**Layer**	Existing (base)	Existing (ABTT)	New (base)	New (ABTT)	Overall (base)	Overall (ABTT)
LaTa	1	0.630	0.899	0.916	0.898	0.738	0.899
	2	0.153	0.899	0.913	0.901	0.439	0.900
	3	**0.256**	**0.907**	**0.842**	**0.907**	**0.477**	**0.907**
	4	0.389	0.920	0.718	0.873	0.513	0.902
	5	0.374	0.914	0.715	0.892	0.502	0.906
	6	0.378	0.910	0.712	0.870	0.503	0.895
	7	0.372	0.890	0.715	0.926	0.501	0.903
	8	0.379	0.877	0.706	0.916	0.502	0.892
	9	0.366	0.888	0.724	0.892	0.501	0.889
	10	0.344	0.884	0.752	0.901	0.498	0.890
	11	0.310	0.907	0.786	0.836	0.490	0.880
	12	0.576	0.880	0.932	0.938	0.710	0.902
PhilTa	**1**	**0.591**	**0.905**	**0.904**	**0.929**	**0.709**	**0.914**
	2	0.479	0.921	0.867	0.842	0.625	0.892
	3	0.193	0.899	0.889	0.879	0.455	0.892
	4	0.148	0.903	0.923	0.858	0.439	0.886
	5	0.252	0.905	0.796	0.885	0.457	0.897
	6	0.338	0.908	0.700	0.901	0.474	0.906
	7	0.301	0.903	0.746	0.910	0.469	0.906
	8	0.262	0.897	0.808	0.867	0.467	0.886
	9	0.303	0.905	0.762	0.879	0.476	0.895
	10	0.299	0.899	0.780	0.870	0.480	0.888
	11	0.256	0.897	0.824	0.867	0.470	0.886
	12	0.338	0.908	0.898	0.882	0.549	0.899
LaBSE	1	0.222	0.897	0.898	0.867	0.477	0.886
	2	0.381	0.880	0.811	0.867	0.543	0.875
	3	0.277	0.912	0.907	0.774	0.514	0.860
	4	0.372	0.907	0.885	0.808	0.565	0.869
	5	0.327	0.854	0.901	0.861	0.543	0.857
	6	0.548	0.875	0.882	0.867	0.674	0.872
	7	0.398	0.869	0.892	0.839	0.584	0.858
	8	0.385	0.865	0.870	0.836	0.568	0.854
	9	0.421	0.886	0.929	0.814	0.612	0.859
	10	0.606	0.884	0.892	0.830	0.713	0.864
	11	**0.785**	**0.929**	**0.913**	**0.842**	**0.833**	**0.896**
	12	0.729	0.895	0.938	0.854	0.808	0.880
Qwen3-0.6B	1	0.264	0.890	0.960	0.941	0.526	0.909
	2	0.437	0.880	0.814	0.916	0.579	0.894
	3	0.250	0.884	0.947	0.929	0.513	0.901
	4	0.411	0.903	0.848	0.907	0.576	0.904
	5	0.181	0.892	0.932	0.935	0.464	0.908
	6	0.422	0.907	0.830	0.904	0.576	0.906
	7	**0.250**	**0.910**	**0.947**	**0.913**	**0.513**	**0.911**
	8	0.409	0.910	0.895	0.913	0.592	0.911
	9	0.460	0.925	0.851	0.885	0.607	0.910
	10	0.533	0.899	0.836	0.901	0.647	0.900
	11	0.293	0.897	0.978	0.885	0.551	0.893
	12	0.407	0.897	0.938	0.904	0.607	0.900
	13	0.432	0.908	0.944	0.879	0.625	0.897
	14	0.456	0.910	0.926	0.864	0.633	0.893
	15	0.469	0.910	0.920	0.864	0.639	0.893
	16	0.639	0.899	0.830	0.848	0.711	0.880
	17	0.480	0.867	0.935	0.892	0.652	0.876
	18	0.553	0.892	0.944	0.867	0.700	0.882
	19	0.514	0.821	0.966	0.898	0.684	0.850
	20	0.654	0.912	0.898	0.833	0.746	0.882
	21	0.647	0.890	0.950	0.895	0.761	0.892
	22	0.731	0.914	0.938	0.858	0.809	0.893

(continued on next page)

Model	Layer	Task A: Directory Assignment					
		Existing (base)	Existing (ABTT)	New (base)	New (ABTT)	Overall (base)	Overall (ABTT)
	23	0.622	0.908	0.966	0.867	0.752	0.893
	24	0.748	0.908	0.935	0.882	0.818	0.899
	25	0.574	0.910	0.975	0.864	0.725	0.893
	26	0.613	0.901	0.975	0.907	0.749	0.903
	27	0.793	0.864	0.833	0.957	0.808	0.899
	28	0.796	0.884	0.867	0.901	0.823	0.890
KaLM-mini	**1**	**0.391**	**0.907**	**0.907**	**0.910**	**0.585**	**0.908**
	2	0.493	0.903	0.848	0.916	0.627	0.908
	3	0.443	0.899	0.870	0.920	0.604	0.907
	4	0.308	0.897	0.935	0.907	0.544	0.901
	5	0.338	0.892	0.882	0.907	0.543	0.897
	6	0.256	0.905	0.910	0.892	0.502	0.900
	7	0.475	0.895	0.854	0.910	0.618	0.901
	8	0.467	0.862	0.870	0.904	0.619	0.878
	9	0.469	0.873	0.885	0.882	0.626	0.876
	10	0.449	0.828	0.885	0.929	0.613	0.866
	11	0.458	0.849	0.885	0.916	0.619	0.874
	12	0.507	0.854	0.895	0.907	0.653	0.874
	13	0.540	0.843	0.895	0.898	0.674	0.864
	14	0.593	0.852	0.845	0.876	0.688	0.861
	15	0.566	0.864	0.898	0.870	0.691	0.866
	16	0.550	0.871	0.935	0.839	0.695	0.859
	17	0.675	0.888	0.920	0.873	0.767	0.882
	18	0.697	0.880	0.960	0.876	0.796	0.879
	19	0.721	0.877	0.966	0.879	0.814	0.878
	20	0.705	0.873	0.966	0.882	0.803	0.876
	21	0.727	0.852	0.966	0.916	0.817	0.876
	22	0.850	0.852	0.916	0.916	0.875	0.876
	23	0.849	0.873	0.920	0.895	0.875	0.881
	24	0.787	0.871	0.944	0.867	0.846	0.869
mT5-base	**1**	**0.135**	**0.908**	**0.981**	**0.904**	**0.453**	**0.907**
	2	0.110	0.908	0.975	0.895	0.436	0.903
	3	0.161	0.892	0.926	0.885	0.449	0.889
	4	0.269	0.824	0.858	0.910	0.491	0.857
	5	0.422	0.770	0.659	0.920	0.512	0.826
	6	0.636	0.832	0.489	0.793	0.580	0.817
	7	0.594	0.819	0.517	0.854	0.565	0.832
	8	0.557	0.843	0.560	0.820	0.558	0.834
	9	0.364	0.845	0.746	0.845	0.508	0.845
	10	0.314	0.832	0.783	0.833	0.491	0.832
	11	0.297	0.791	0.780	0.858	0.479	0.816
	12	0.198	0.841	0.926	0.885	0.472	0.858

Table 2 reports per-layer Task A assignment metrics for each model, comparing the uncorrected `baseline` (mean pooling) against `sif_abtt_optimal` (SIF weighting plus ABTT with D tuned per layer on the train split). Bolded rows mark each model's best layer by ABTT overall assignment accuracy. The two numbers drift apart sharply in the middle layers of the Latin T5 encoders (LaTa, PhilTa) and in `mt5-base`, and much less in LaBSE, Qwen3-0.6B, and KaLM-mini. The remainder of this section unpacks where these numbers come from layer by layer.

4.2 The Mid-Layer Dip

The main result shows up immediately in the layerwise plots. Baseline mean-pooled embeddings from LaTa and PhilTa collapse in the middle encoder layers. LaTa drops from strong early-layer AUCROC to below 0.5, which is chance, and PhilTa follows almost the same path. Both recover somewhat by the final layer.

LaBSE and mt5-base do not show the same kind of collapse, and even their weakest layers remain well above chance. Qwen3-0.6B and KaLM-mini also vary much less across depth, which fits their decoder-only architecture.

After ABTT, the scores within each model cluster much more tightly across depth.

4.3 Similarity Distributions

Figure 1 shows the same pattern from another angle. It compares PhilTa at the last layer and at its dip layer. Before ABTT, the dip-layer cosine scores for true duplicates and non-duplicates overlap heavily. After ABTT, those distributions separate much more clearly at the same layer.

4.4 Cosine Gap Analysis

The cosine-gap plots tell the same story in a more compact way. In LaTa and PhilTa, the baseline gap nearly disappears in the dip region. ABTT restores a clear positive gap across all layers.

4.5 Task B: Simulated Human Selection

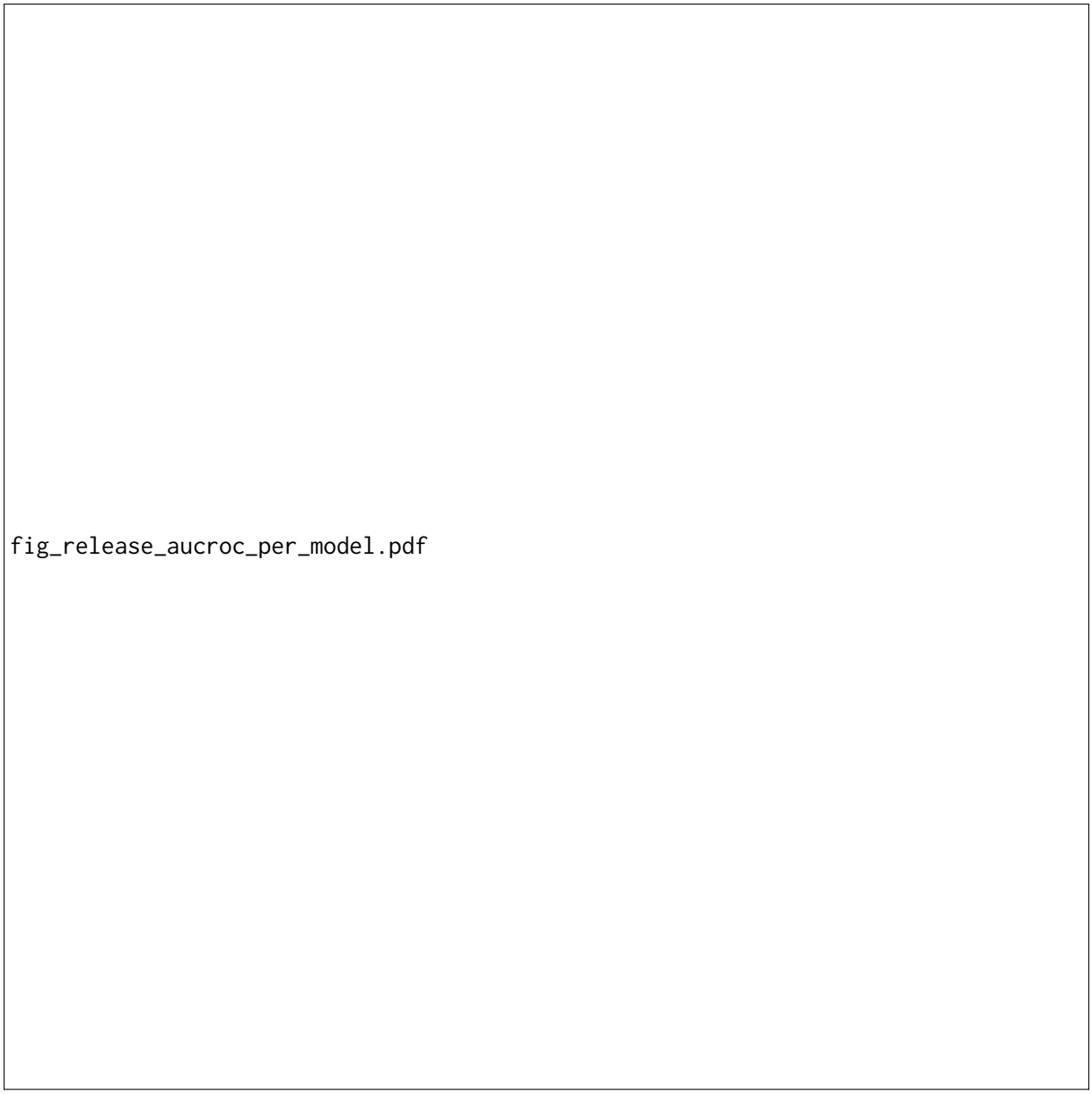

Figure 2: AUCROC across normalized layer depth for the six models. LaTa and PhilTa show the clearest mid-layer dip before post-processing. ABTT flattens those curves much more than SIF or whitening.

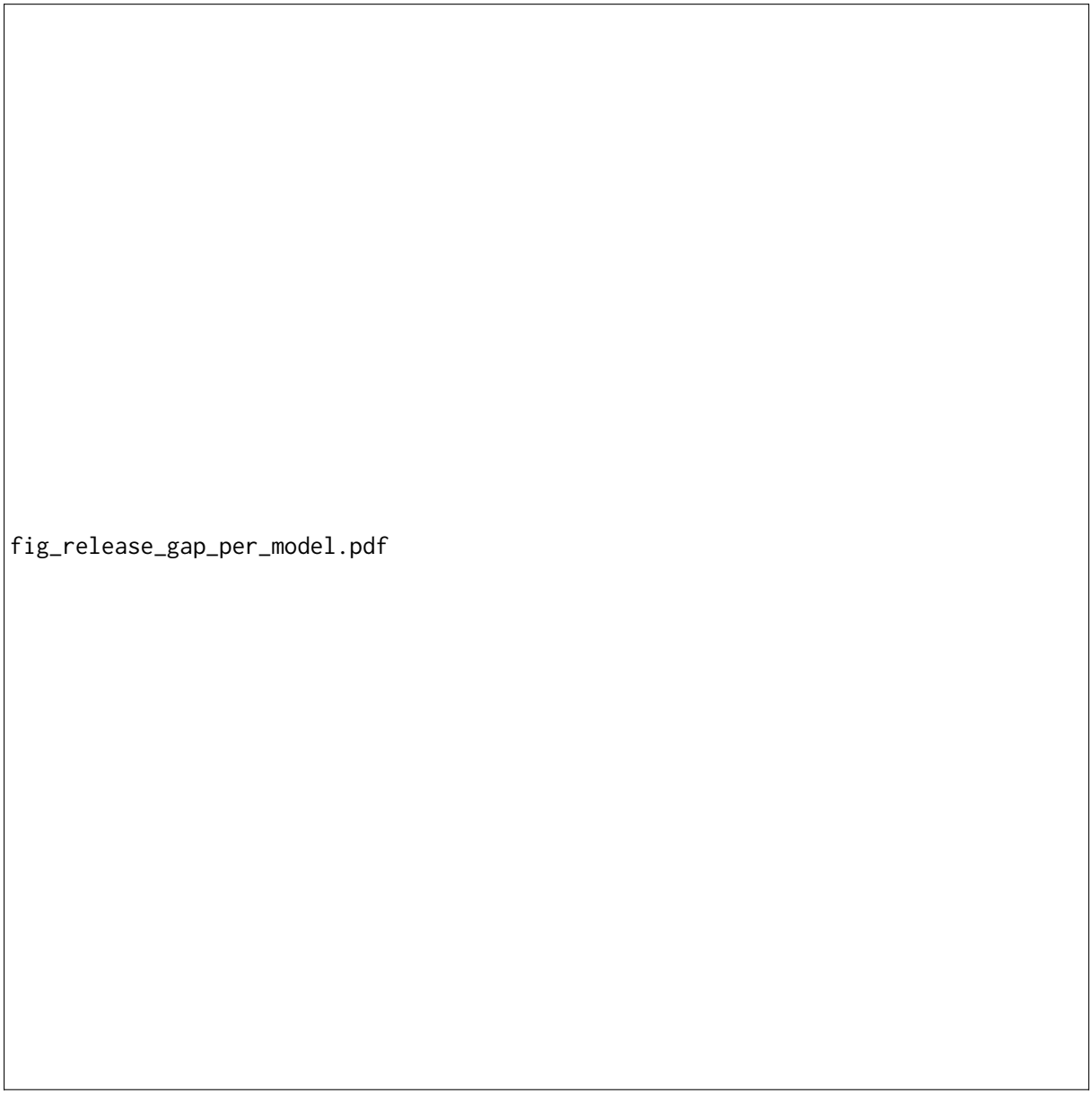

Figure 3: Cosine gap across normalized layer depth for the six models. In the Latin T5 models, the baseline gap nearly vanishes in the middle layers. ABTT restores a positive gap across depth more reliably than the alternative baselines.

Table 3: Per-layer Task B accuracy for each model, averaged over $M = 5$ query/reference reseedings of the v2 train/test split (mean $\pm$ std). `baseline` is mean pooling without correction; `sif_abtt_optimal` applies SIF weighting plus ABTT with D tuned per layer on the train split. Overall assignment accuracy coincides with Acc@1 in the Task B mseed pipeline (assignment = top-1 pick), so we instead report the existing vs. new decomposition. Rows in bold mark the best layer per model, selected by ABTT Acc@1.

		Task B: Simulated Human Selection					
Model	**Layer**	Acc@1 (base)	Acc@1 (ABTT)	Existing (base)	Existing (ABTT)	New (base)	New (ABTT)
LaTa	1	0.731 ± 0.009	0.889 ± 0.009	0.562 ± 0.004	0.842 ± 0.009	0.949 ± 0.011	0.950 ± 0.011
	2	0.428 ± 0.011	0.887 ± 0.005	0.048 ± 0.006	0.840 ± 0.005	0.920 ± 0.008	0.949 ± 0.009
	3	0.392 ± 0.012	0.889 ± 0.008	0.046 ± 0.008	0.845 ± 0.006	0.840 ± 0.020	0.946 ± 0.013
	4	0.330 ± 0.018	0.888 ± 0.008	0.047 ± 0.008	0.858 ± 0.007	0.696 ± 0.030	0.927 ± 0.013
	5	0.326 ± 0.020	0.890 ± 0.008	0.039 ± 0.004	0.856 ± 0.005	0.698 ± 0.028	0.935 ± 0.014
	6	0.321 ± 0.018	0.890 ± 0.008	0.038 ± 0.007	0.861 ± 0.008	0.688 ± 0.026	0.927 ± 0.014
	7	**0.327 ± 0.020**	**0.895 ± 0.006**	**0.044 ± 0.007**	**0.846 ± 0.006**	**0.694 ± 0.029**	**0.957 ± 0.010**
	8	0.325 ± 0.022	0.883 ± 0.007	0.046 ± 0.010	0.832 ± 0.006	0.685 ± 0.032	0.949 ± 0.011
	9	0.331 ± 0.019	0.885 ± 0.008	0.045 ± 0.010	0.846 ± 0.006	0.701 ± 0.024	0.935 ± 0.013
	10	0.346 ± 0.017	0.888 ± 0.002	0.044 ± 0.010	0.844 ± 0.007	0.736 ± 0.022	0.944 ± 0.010
	11	0.360 ± 0.018	0.881 ± 0.006	0.045 ± 0.009	0.860 ± 0.008	0.767 ± 0.028	0.907 ± 0.011
	12	0.707 ± 0.015	0.889 ± 0.005	0.511 ± 0.017	0.834 ± 0.005	0.961 ± 0.008	0.961 ± 0.010
PhilTa	1	0.706 ± 0.016	0.898 ± 0.007	0.532 ± 0.018	0.856 ± 0.009	0.931 ± 0.012	0.953 ± 0.011
	2	0.625 ± 0.018	0.889 ± 0.009	0.408 ± 0.016	0.866 ± 0.009	0.905 ± 0.017	0.919 ± 0.017
	3	0.439 ± 0.013	0.892 ± 0.006	0.071 ± 0.014	0.856 ± 0.008	0.917 ± 0.013	0.939 ± 0.009
	4	0.420 ± 0.016	0.893 ± 0.009	0.027 ± 0.007	0.858 ± 0.006	0.928 ± 0.015	0.938 ± 0.015
	5	0.374 ± 0.015	0.896 ± 0.010	0.042 ± 0.011	0.872 ± 0.007	0.805 ± 0.018	0.925 ± 0.014
	6	0.338 ± 0.015	0.897 ± 0.008	0.059 ± 0.010	0.861 ± 0.007	0.698 ± 0.025	0.945 ± 0.014
	7	**0.361 ± 0.016**	**0.900 ± 0.009**	**0.056 ± 0.010**	**0.859 ± 0.005**	**0.757 ± 0.021**	**0.952 ± 0.016**
	8	0.390 ± 0.016	0.891 ± 0.010	0.058 ± 0.012	0.858 ± 0.007	0.820 ± 0.017	0.933 ± 0.018
	9	0.372 ± 0.015	0.894 ± 0.006	0.066 ± 0.012	0.861 ± 0.006	0.768 ± 0.026	0.936 ± 0.011
	10	0.377 ± 0.013	0.892 ± 0.006	0.066 ± 0.012	0.855 ± 0.003	0.781 ± 0.021	0.941 ± 0.015
	11	0.394 ± 0.013	0.889 ± 0.006	0.058 ± 0.008	0.851 ± 0.008	0.829 ± 0.018	0.937 ± 0.008
	12	0.563 ± 0.010	0.896 ± 0.007	0.279 ± 0.011	0.863 ± 0.009	0.932 ± 0.005	0.938 ± 0.014
LaBSE	1	0.501 ± 0.011	0.858 ± 0.008	0.170 ± 0.002	0.838 ± 0.007	0.930 ± 0.014	0.884 ± 0.010
	2	0.522 ± 0.010	0.861 ± 0.006	0.272 ± 0.003	0.817 ± 0.004	0.844 ± 0.022	0.917 ± 0.013
	3	0.535 ± 0.008	0.845 ± 0.005	0.217 ± 0.007	0.831 ± 0.010	0.946 ± 0.007	0.864 ± 0.009
	4	0.568 ± 0.008	0.849 ± 0.007	0.298 ± 0.006	0.837 ± 0.008	0.918 ± 0.014	0.865 ± 0.015
	5	0.550 ± 0.011	0.845 ± 0.007	0.254 ± 0.011	0.794 ± 0.006	0.933 ± 0.013	0.911 ± 0.015
	6	0.660 ± 0.012	0.858 ± 0.003	0.460 ± 0.021	0.816 ± 0.009	0.918 ± 0.018	0.912 ± 0.016
	7	0.595 ± 0.010	0.850 ± 0.003	0.333 ± 0.019	0.808 ± 0.006	0.934 ± 0.015	0.904 ± 0.010
	8	0.571 ± 0.013	0.848 ± 0.004	0.313 ± 0.019	0.802 ± 0.011	0.905 ± 0.018	0.906 ± 0.007
	9	0.630 ± 0.008	0.855 ± 0.004	0.380 ± 0.018	0.828 ± 0.005	0.954 ± 0.008	0.890 ± 0.007
	10	0.711 ± 0.015	0.856 ± 0.003	0.550 ± 0.018	0.835 ± 0.005	0.920 ± 0.016	0.884 ± 0.004
	11	**0.820 ± 0.009**	**0.904 ± 0.008**	**0.732 ± 0.011**	**0.873 ± 0.010**	**0.935 ± 0.013**	**0.944 ± 0.017**
	12	0.792 ± 0.013	0.883 ± 0.006	0.668 ± 0.016	0.849 ± 0.009	0.952 ± 0.013	0.928 ± 0.014
Qwen3-0.6B	1	0.549 ± 0.006	0.892 ± 0.004	0.221 ± 0.009	0.865 ± 0.008	0.972 ± 0.003	0.927 ± 0.007
	2	0.570 ± 0.013	0.882 ± 0.003	0.362 ± 0.012	0.828 ± 0.007	0.839 ± 0.028	0.951 ± 0.006
	3	0.540 ± 0.006	0.889 ± 0.005	0.214 ± 0.010	0.827 ± 0.011	0.962 ± 0.007	0.969 ± 0.011
	4	0.581 ± 0.010	0.891 ± 0.004	0.354 ± 0.008	0.844 ± 0.009	0.874 ± 0.023	0.952 ± 0.010
	5	0.486 ± 0.016	0.894 ± 0.007	0.133 ± 0.013	0.839 ± 0.011	0.943 ± 0.012	0.965 ± 0.010
	6	0.566 ± 0.011	0.897 ± 0.005	0.349 ± 0.009	0.857 ± 0.010	0.848 ± 0.020	0.949 ± 0.013
	7	0.540 ± 0.007	0.902 ± 0.006	0.211 ± 0.011	0.862 ± 0.009	0.967 ± 0.012	0.955 ± 0.015
	8	**0.608 ± 0.010**	**0.903 ± 0.006**	**0.361 ± 0.016**	**0.861 ± 0.011**	**0.927 ± 0.015**	**0.958 ± 0.009**
	9	0.610 ± 0.011	0.901 ± 0.005	0.391 ± 0.015	0.867 ± 0.009	0.892 ± 0.019	0.944 ± 0.014
	10	0.636 ± 0.012	0.890 ± 0.008	0.450 ± 0.009	0.847 ± 0.014	0.877 ± 0.020	0.946 ± 0.010
	11	0.577 ± 0.006	0.886 ± 0.005	0.257 ± 0.013	0.838 ± 0.016	0.991 ± 0.005	0.948 ± 0.015
	12	0.626 ± 0.009	0.893 ± 0.004	0.365 ± 0.019	0.844 ± 0.013	0.964 ± 0.013	0.958 ± 0.010
	13	0.638 ± 0.010	0.888 ± 0.006	0.382 ± 0.016	0.849 ± 0.013	0.970 ± 0.015	0.938 ± 0.009
	14	0.644 ± 0.010	0.885 ± 0.007	0.401 ± 0.019	0.852 ± 0.012	0.958 ± 0.015	0.929 ± 0.008
	15	0.646 ± 0.013	0.885 ± 0.005	0.409 ± 0.020	0.852 ± 0.012	0.952 ± 0.016	0.929 ± 0.017
	16	0.689 ± 0.015	0.873 ± 0.006	0.544 ± 0.017	0.837 ± 0.010	0.877 ± 0.016	0.920 ± 0.020
	17	0.657 ± 0.010	0.861 ± 0.008	0.419 ± 0.020	0.807 ± 0.014	0.964 ± 0.012	0.932 ± 0.017
	18	0.708 ± 0.011	0.868 ± 0.004	0.503 ± 0.019	0.833 ± 0.016	0.973 ± 0.005	0.913 ± 0.020
	19	0.690 ± 0.011	0.873 ± 0.005	0.462 ± 0.018	0.859 ± 0.008	0.986 ± 0.006	0.892 ± 0.015
	20	0.742 ± 0.019	0.880 ± 0.006	0.588 ± 0.023	0.858 ± 0.009	0.941 ± 0.017	0.909 ± 0.016

(continued on next page)

Model	Layer	Task B: Simulated Human Selection					
		Acc@1 (base)	Acc@1 (ABTT)	Existing (base)	Existing (ABTT)	New (base)	New (ABTT)
	21	0.757 ± 0.016	0.878 ± 0.006	0.584 ± 0.021	0.835 ± 0.009	0.981 ± 0.010	0.934 ± 0.013
	22	0.798 ± 0.014	0.882 ± 0.006	0.670 ± 0.021	0.854 ± 0.004	0.965 ± 0.006	0.917 ± 0.017
	23	0.757 ± 0.013	0.888 ± 0.007	0.578 ± 0.016	0.857 ± 0.005	0.989 ± 0.004	0.929 ± 0.016
	24	0.810 ± 0.014	0.894 ± 0.005	0.694 ± 0.022	0.861 ± 0.005	0.960 ± 0.003	0.936 ± 0.015
	25	0.732 ± 0.012	0.890 ± 0.009	0.528 ± 0.016	0.866 ± 0.005	0.997 ± 0.003	0.922 ± 0.017
	26	0.749 ± 0.013	0.895 ± 0.007	0.561 ± 0.017	0.858 ± 0.002	0.991 ± 0.003	0.943 ± 0.017
	27	0.811 ± 0.004	0.881 ± 0.003	0.741 ± 0.008	0.824 ± 0.007	0.901 ± 0.011	0.954 ± 0.011
	28	0.822 ± 0.010	0.870 ± 0.007	0.733 ± 0.014	0.800 ± 0.012	0.937 ± 0.006	0.961 ± 0.009
KaLM-mini	1	0.595 ± 0.011	0.893 ± 0.004	0.344 ± 0.002	0.846 ± 0.007	0.920 ± 0.021	0.954 ± 0.009
	2	$\mathbf{0.620 \pm 0.017}$	$\mathbf{0.894 \pm 0.005}$	$\mathbf{0.422 \pm 0.014}$	$\mathbf{0.845 \pm 0.005}$	$\mathbf{0.875 \pm 0.021}$	$\mathbf{0.958 \pm 0.013}$
	3	0.604 ± 0.010	0.894 ± 0.003	0.378 ± 0.009	0.843 ± 0.003	0.898 ± 0.017	0.960 ± 0.011
	4	0.562 ± 0.015	0.889 ± 0.004	0.265 ± 0.009	0.842 ± 0.010	0.946 ± 0.016	0.951 ± 0.011
	5	0.550 ± 0.012	0.890 ± 0.005	0.270 ± 0.007	0.841 ± 0.006	0.912 ± 0.018	0.954 ± 0.012
	6	0.522 ± 0.015	0.894 ± 0.004	0.202 ± 0.011	0.850 ± 0.006	0.935 ± 0.013	0.950 ± 0.015
	7	0.604 ± 0.017	0.894 ± 0.003	0.394 ± 0.016	0.840 ± 0.011	0.876 ± 0.020	0.963 ± 0.012
	8	0.607 ± 0.013	0.866 ± 0.003	0.387 ± 0.010	0.798 ± 0.010	0.892 ± 0.016	0.955 ± 0.010
	9	0.614 ± 0.010	0.874 ± 0.005	0.386 ± 0.009	0.821 ± 0.011	0.909 ± 0.013	0.944 ± 0.011
	10	0.602 ± 0.012	0.860 ± 0.007	0.359 ± 0.005	0.774 ± 0.012	0.917 ± 0.017	0.970 ± 0.012
	11	0.601 ± 0.012	0.864 ± 0.008	0.360 ± 0.009	0.792 ± 0.012	0.911 ± 0.020	0.957 ± 0.008
	12	0.638 ± 0.009	0.868 ± 0.006	0.415 ± 0.008	0.801 ± 0.009	0.927 ± 0.016	0.955 ± 0.005
	13	0.643 ± 0.010	0.862 ± 0.006	0.435 ± 0.007	0.792 ± 0.005	0.912 ± 0.016	0.953 ± 0.016
	14	0.653 ± 0.009	0.858 ± 0.007	0.479 ± 0.002	0.793 ± 0.009	0.879 ± 0.016	0.943 ± 0.009
	15	0.659 ± 0.007	0.868 ± 0.005	0.458 ± 0.006	0.812 ± 0.009	0.919 ± 0.010	0.941 ± 0.006
	16	0.682 ± 0.005	0.869 ± 0.008	0.475 ± 0.006	0.824 ± 0.008	0.951 ± 0.010	0.926 ± 0.009
	17	0.756 ± 0.007	0.885 ± 0.009	0.611 ± 0.018	0.841 ± 0.006	0.944 ± 0.010	0.942 ± 0.015
	18	0.780 ± 0.010	0.881 ± 0.007	0.635 ± 0.016	0.840 ± 0.002	0.967 ± 0.007	0.933 ± 0.016
	19	0.804 ± 0.013	0.876 ± 0.007	0.672 ± 0.022	0.831 ± 0.007	0.977 ± 0.003	0.935 ± 0.008
	20	0.793 ± 0.011	0.877 ± 0.004	0.649 ± 0.020	0.831 ± 0.003	0.979 ± 0.004	0.936 ± 0.008
	21	0.804 ± 0.011	0.871 ± 0.007	0.672 ± 0.018	0.830 ± 0.005	0.975 ± 0.006	0.924 ± 0.012
	22	0.870 ± 0.008	0.873 ± 0.004	0.810 ± 0.007	0.812 ± 0.005	0.948 ± 0.008	0.952 ± 0.009
	23	0.873 ± 0.007	0.884 ± 0.004	0.811 ± 0.008	0.833 ± 0.004	0.953 ± 0.007	0.951 ± 0.007
	24	0.848 ± 0.011	0.856 ± 0.005	0.749 ± 0.019	0.806 ± 0.009	0.975 ± 0.007	0.921 ± 0.010
mT5-base	**1**	$\mathbf{0.486 \pm 0.012}$	$\mathbf{0.898 \pm 0.004}$	$\mathbf{0.100 \pm 0.007}$	$\mathbf{0.856 \pm 0.009}$	$\mathbf{0.987 \pm 0.007}$	$\mathbf{0.953 \pm 0.010}$
	2	0.475 ± 0.012	0.892 ± 0.004	0.082 ± 0.008	0.851 ± 0.013	0.984 ± 0.008	0.946 ± 0.013
	3	0.482 ± 0.014	0.872 ± 0.008	0.123 ± 0.008	0.809 ± 0.012	0.947 ± 0.008	0.954 ± 0.008
	4	0.498 ± 0.013	0.852 ± 0.006	0.205 ± 0.006	0.772 ± 0.012	0.879 ± 0.018	0.956 ± 0.010
	5	0.370 ± 0.013	0.825 ± 0.009	0.155 ± 0.005	0.724 ± 0.010	0.649 ± 0.020	0.956 ± 0.005
	6	0.359 ± 0.011	0.809 ± 0.006	0.263 ± 0.021	0.765 ± 0.013	0.483 ± 0.018	0.865 ± 0.018
	7	0.368 ± 0.007	0.827 ± 0.007	0.256 ± 0.012	0.758 ± 0.011	0.512 ± 0.018	0.916 ± 0.012
	8	0.380 ± 0.007	0.824 ± 0.012	0.245 ± 0.007	0.771 ± 0.013	0.555 ± 0.017	0.892 ± 0.015
	9	0.426 ± 0.012	0.836 ± 0.010	0.171 ± 0.006	0.781 ± 0.014	0.756 ± 0.023	0.907 ± 0.006
	10	0.420 ± 0.007	0.821 ± 0.016	0.141 ± 0.011	0.765 ± 0.016	0.782 ± 0.019	0.893 ± 0.017
	11	0.418 ± 0.012	0.803 ± 0.010	0.132 ± 0.010	0.719 ± 0.013	0.788 ± 0.014	0.910 ± 0.010
	12	0.506 ± 0.008	0.840 ± 0.009	0.170 ± 0.008	0.770 ± 0.017	0.943 ± 0.012	0.930 ± 0.008

We next return to the directory-assignment setting that motivates deployment. For each query, the system scores every directory by its best document match and then presents the top five options, along with the implicit `__NEW__` fallback. During evaluation, the "human" is simulated by selecting the labeled directory whenever it appears in the shortlist. Table 3 reports per-layer Acc@1 and the existing-vs-new decomposition, averaged over $M = 5$ query/reference reseedings. Table 4 then reports cumulative top-K accuracy at each model's best layer: the fraction of queries whose labeled directory appears within the top K options, for $K = 1, \ldots, 5$.

The cumulative view directly answers the deployment question: if the expert inspects the top K candidates, how likely are they to find the correct directory? All six models place the labeled directory in the top-1 position at least 89.4% of the time averaged over five seeds, and by Top-2 every model exceeds 95%. Seed-to-seed standard deviations stay at or below ± 1.1 points, so the ranking of models is stable rather than dependent on the query/reference split. The marginal gains from Top-3 onward are small, suggesting that a shortlist of two or three options suffices for nearly all queries.

4.6 Example-Level Explanations

Integrated gradients are local explanations, so we focus on concrete examples rather than global summaries. For a query passage and one candidate partner passage, we compute per-sequence integrated gradients toward pair cosine similarity and display the original decoded tokens on both axes. Figure 4 gives the main explanation view, a derived pairwise heatmap built from token-token cosine similarity and weighted by the marginal integrated-gradient scores of the two passages.

The pairwise heatmap carries most of the interpretive weight. It is meant to help a human decide whether the retrieved directory looks plausible, not to stand in for a global account of model behavior. Figure 5 adds one more view of the same example.

The attention map provides extra context around the same query and candidate pair. It is not the explanation itself, and it is not meant to make a before-and-after claim about ABTT.

4.7 Cross-Dataset Validation: Decritals

To check that our findings are not an artifact of a single corpus, we re-ran the same six models, the same extraction pipeline, and the same v2 split protocol on *decritals*, a second Medieval Latin manuscript collection. Decritals is sparser than canon: of 226 test files, only 191 have a same-directory partner in test, leaving 35 "new" files and a positive class that is small in absolute terms.

Two patterns are worth flagging in Table 5. First, five of six models converge to an overall assignment accuracy of exactly 0.845, with KaLM-mini one prediction beyond at 0.850. The 0.845 number is not a coincidence: $191/226 = 0.8451$ is the rate of the all-existing degenerate classifier on this test set. With only 35 "new" files in the test set, the F_1-optimal threshold τ collapses toward the lower end of the cosine range, and the assignment decision degenerates to "always existing". This is exactly the failure mode that Section 3.6 anticipated: when τ is poorly calibrated against a sparse positive set, assignment accuracy and rank-1 directory accuracy diverge sharply, and the threshold-free Acc@1 is the more informative metric.

Second, threshold-free Acc@1 itself drops by 0.34–0.38 absolute from canon to decritals across all six models, with no model breaking out of the 0.500–0.544 band. The relative ordering shifts only mildly (PhilTa edges out the rest, mt5-base trails); the much larger movement is the collective drop in absolute retrieval quality. This says that the binding bottleneck on decritals is retrieval quality on a harder collection, not threshold calibration: even on the metric that is immune to τ, every model loses roughly the same large fraction of its canon performance. The qualitative dip-and-repair pattern from canon does survive the transfer (best-layer choices for ABTT-cleaned representations still improve over baselines on a per-layer basis), but the absolute floor moves down enough that the canon-level deployment story does not extend to decritals without further work.

5 Discussion

PCA whitening has worked well for English sentence similarity (Su et al., 2021; Huang et al., 2021), but it performs poorly here. A plausible reason is that whitening removes variance that still helps separate equivalent from non-equivalent Latin legal passages. In this corpus, the original variance structure seems to carry useful signal rather than just noise.

The depth profiles also differ by architecture. The strongest dip appears in the two Latin T5 encoders. mt5-base shows a milder version of the

Model	Top-1	Top-2	Top-3	Top-4	Top-5
LaTa	90.1 ± 1.1	96.0 ± 0.4	97.1 ± 0.1	97.6 ± 0.4	97.7 ± 0.4
PhilTa	90.0 ± 0.9	95.7 ± 0.4	96.9 ± 0.4	97.4 ± 0.3	97.7 ± 0.2
LaBSE	90.4 ± 0.8	95.5 ± 0.4	96.4 ± 0.3	97.1 ± 0.5	97.7 ± 0.4
Qwen3-0.6B	90.3 ± 0.6	95.5 ± 0.5	96.8 ± 0.3	97.2 ± 0.5	97.4 ± 0.4
KaLM-mini	89.4 ± 0.5	95.1 ± 0.4	96.4 ± 0.2	96.9 ± 0.3	97.3 ± 0.4
mt5-base	89.8 ± 0.4	95.6 ± 0.5	96.6 ± 0.3	96.9 ± 0.2	97.2 ± 0.2

Table 4: Cumulative top-K accuracy for Task B across the six models, reported as mean $\pm$ standard deviation over $M = 5$ query/reference reseedings at a fixed v2 train/test split. Each cell gives the percentage of test queries whose labeled directory appears within the model's top K options at its best layer/method configuration. All models exceed 89% by Top-1 and 95% by Top-2.

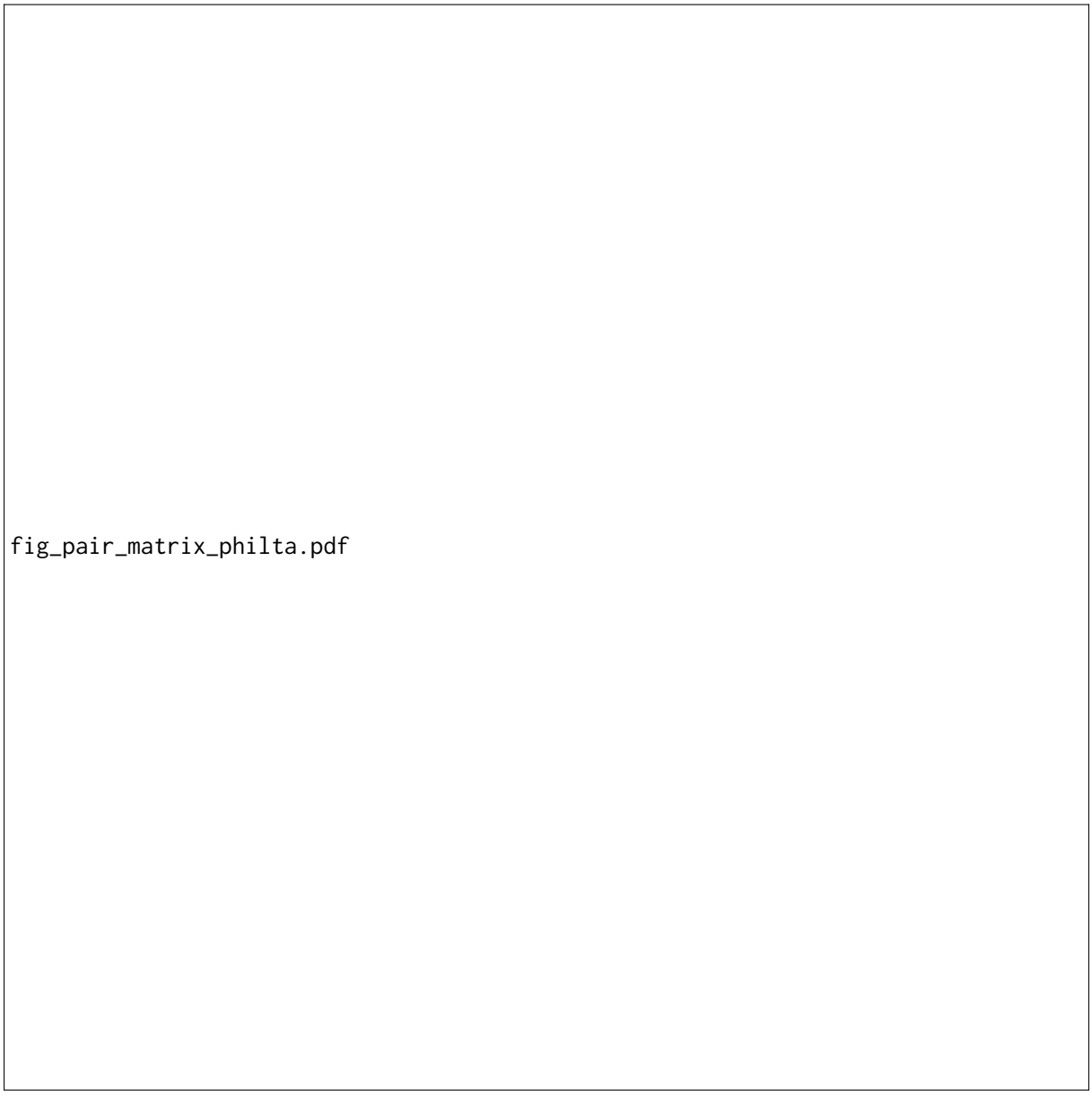

Figure 4: Derived pairwise heatmap for one PhilTa retrieval case. The left panel shows the baseline representation and the right panel shows the same example after ABTT. Both axes use original decoded tokens, and the cell values combine token-token cosine similarity with single-sequence integrated-gradient scores.

same pattern, and LaBSE plus the decoder-only models vary much less across depth. Architecture and pre-training objective are the most likely reasons. After ABTT, those differences matter less in

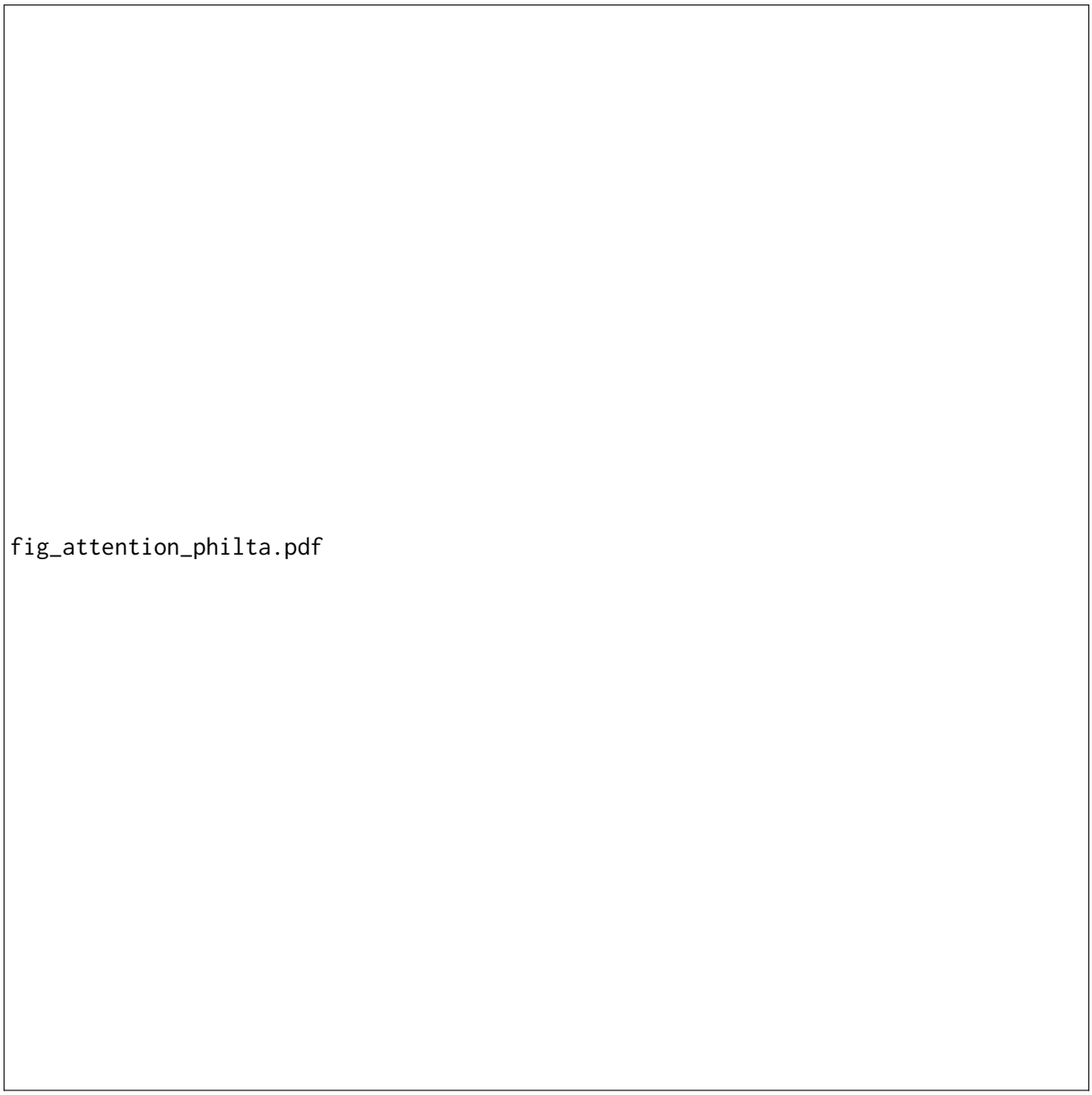

Figure 5: Companion self-attention maps for the same PhilTa example. Because ABTT is applied after encoding, attention is included only as supporting context, not as a before-and-after comparison.

practice because the weak middle layers become much more usable. It also reduces the need to hunt for a narrow "good" layer band before retrieval.

We also tested SIF-weighted pooling and whitening as simpler alternatives. They can help in individual cases, but neither produces the same consistent repair across models and layers. Mean pooling plus ABTT remains the safer default for this retrieval pipeline.

The explanation component is intentionally local. The derived pair matrix and the companion attention map are there to help a scholar inspect one ranked option, not to summarize the model as a whole. That fits the human-in-the-loop directory-assignment setting much better than our earlier aggregate token-category analyses. We also ran the before-and-after ABTT comparison using five additional attribution methods beyond integrated gradients on the same example pairs, with consistent improvements in attribution clarity across all six models; for brevity we report only the IG pair-matrix view in the main text.

Limitations

Our study covers one domain, medieval canon law, and one language, Latin. The 1,705-fragment corpus is large for this field, but it is still small by current NLP standards. We test six models, and

Model	Acc@1	AssignAcc	TaskB Acc@1
LaTa	0.535	0.845	0.466 ± 0.022
PhilTa	0.544	0.845	0.471 ± 0.034
LaBSE	0.527	0.845	0.436 ± 0.039
Qwen3-0.6B	0.531	0.845	0.435 ± 0.018
KaLM-mini	0.535	**0.850**	0.428 ± 0.028
mt5-base	0.500	0.845	0.406 ± 0.029

Table 5: Decritals cross-dataset results. Acc@1 is the threshold-free directory accuracy at rank 1 on Task A. AssignAcc is the overall assignment accuracy at the best per-model layer. TaskB Acc@1 reports the directory rank-1 accuracy from the M=5 query/reference reseeding (mean $\pm$ std).

models outside this set may behave differently. ABTT also needs enough unlabeled documents to estimate principal components well, and we have not tested it in settings with only a few hundred texts or fewer. The explanation analysis is local and example-based, and the human-in-the-loop results are simulated from ground truth rather than validated through a live user study. As Section 4.7 shows, transferring the same protocol to a sparser collection (decritals) preserves the qualitative dip-and-repair behaviour but degrades absolute retrieval quality and collapses the F_1-tuned threshold to a degenerate all-existing baseline; the cross-collection generalization story is therefore qualified rather than complete.

Our example-level attributions use vanilla integrated gradients, which produces a single token saliency vector per sequence and does not jointly reason about the query–candidate pair. Following recent learned-mask attribution work such as MaRC (Brinner and Zarrieß, 2023), we plan to jointly optimize soft masks on both passages against the ABTT-cleaned cosine similarity, tying the attribution objective directly to the same geometry the retrieval pipeline relies on. Empirical evaluation of this learned-mask variant is left to future work.

References

Sanjeev Arora, Yingyu Liang, and Tengyu Ma. 2017. A simple but tough-to-beat baseline for sentence embeddings. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*.

Marc Brinner and Sina Zarrieß. 2023. Model interpretability and rationale extraction by input mask optimization. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13722–13744. Association for Computational Linguistics.

Fangxiaoyu Feng, Yinfei Yang, Daniel Cer, Naveen Arivazhagan, and Wei Wang. 2022. Language-agnostic BERT sentence embedding. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 878–891. Association for Computational Linguistics.

Junjie Huang, Duyu Tang, Wanjun Zhong, Shuai Lu, Linjun Shou, Ming Gong, Daxin Jiang, and Nan Duan. 2021. WhiteningBERT: An easy unsupervised sentence embedding approach. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 238–244. Association for Computational Linguistics.

Xinyu Lu, Nurlan Maharramov, and 1 others. 2025. KaLM-embedding: Superior training data brings a stronger embedding model. Accessed: 2025.

Jiaqi Mu and Pramod Viswanath. 2018. All-but-the-top: Simple and effective postprocessing for word representations. In *Proceedings of the 6th International Conference on Learning Representations (ICLR)*.

Qwen Team. 2025. Qwen3-embedding: Advancing text embedding and reranking through foundation models. Accessed: 2025.

Frederick Riemenschneider and Anette Frank. 2023. Exploring large language models for classical philology. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15181–15199. Association for Computational Linguistics.

Jianlin Su, Jiarun Cao, Weijie Liu, and Yangyiwen Ou. 2021. Whitening sentence representations for better semantics and faster retrieval. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 3319–3328. PMLR.

Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 483–498. Association for Computational Linguistics.